

DESIGN AND ANALYSIS OF ALGORITHMS PROJECT REPORT

Syed Huzaifa Ahmed 21K-3403, Daniyal Ahmed Shah 20K-1691

November 26, 2023

Abstract:

In this project, we examined 5 Geometric Algorithms and 3 Line Segment Intersection algorithms. We conducted a thorough analysis of their complexities to determine which algorithm would offer the most advantages for future work involving convex hulls and segments in real-world applications.

1 Introduction:

We worked upon Brute force, Graham Scan, Jarvis March, Quick Hull, Monotone Chain algorithms for computing the convex hull. In line intersection we took counter clock wise test, line segment intersection, line sweep algorithm for checking the intersection of two lines. The main objective was to compute as well as compare different convex hull algorithms as well as line intersection algorithms

2 Programming Design:

Python 3.12.0 is the programming language used to accomplish our objective. A list of 2 dimensional points are considered in both the two different sets of algorithms. The user can either create a file or use the interface for a quick review of the implemented algorithm.

3 Experimental Setup:

An interface will be drawn in both the ways of entering inputs. In intersection algorithms upon entering all the

four points the screen will halt and on pressing key 'a', the output will be shown, Execution time can be noted from the terminal where as time and space complexities will be shown on the interface, Similar is the case for convex hull algorithms input will be accepted on pressing 'Enter' key. A "Pygame" library is being used for the GUI and the IDE is Visual Studio Code.

4 Results and Discussions:

Following are the screenshots of the GUI output of the 3 line intersection algorithms upon random user input

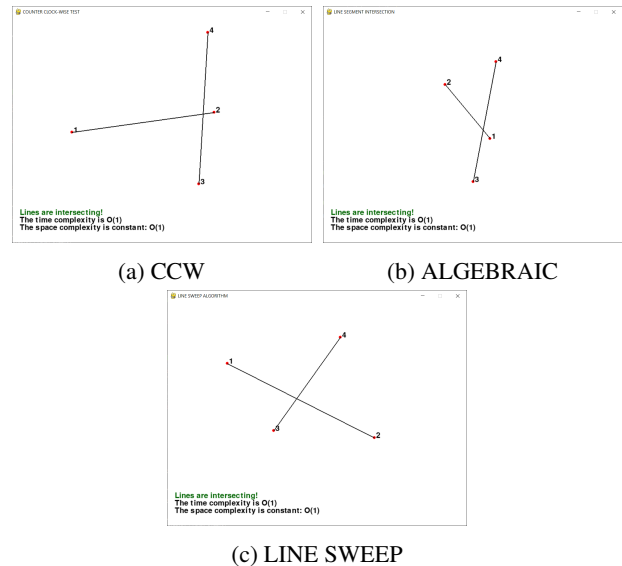


Figure 1: LINE INTERSECTION ALGORITHMS

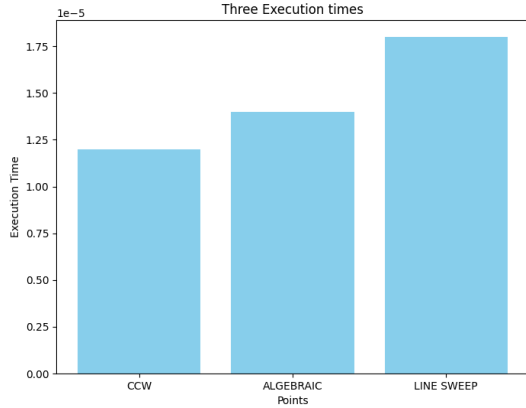


Figure 2: EXECUTION TIMES

We recorded the finding of the following three execution times and represented them using the above bar graph. This states that there is not much of a difference between execution time of the three algorithms.

Following are the screenshots of the GUI output of the 5 convex hull algorithms upon random user input

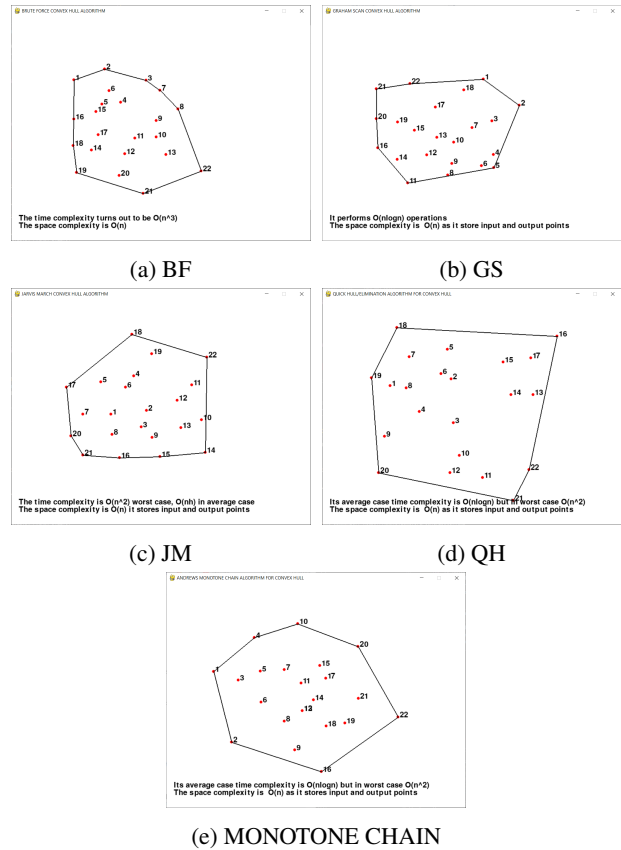


Figure 3: LINE INTERSECTION ALGORITHMS

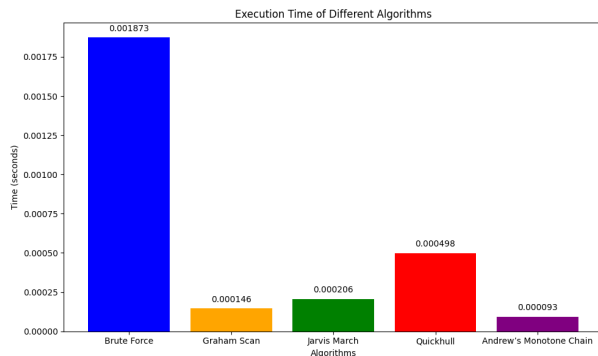


Figure 4: EXECUTION TIMES CONVEX HULLS

We recorded the finding of the following five execution times and represented them using the above bar graph. We can clearly see the difference in execution time among brute force and the other four algorithms.

5 Conclusion:

For small datasets or when simplicity is more important than efficiency, Jarvis March, or Graham Scan can be considered. Brute Force is tend to be avoided due to its higher time complexity. For larger datasets, Graham Scan, Monotone Chain, or Quick Hull might be preferable due to their better average time complexity. The choice of algorithm often depends on the specific characteristics of the dataset (size, distribution, etc.) and the balance between simplicity and efficiency required for the application. Whereas in line intersection, CCW provides a basic orientation test that's used in line intersection algorithms but might not be suitable for larger datasets. Line Sweep is highly efficient for detecting intersections among line segments in large datasets. Algebraic approaches provide precision but might have limitations with specific line configurations and could be computationally intensive. The choice of algorithm depends on the dataset's size, precision requirements, and specific characteristics of the lines being analyzed.

[1] [2]

References

- [1] Kurt Mehlhorn and Stefan Näher. Implementation of a sweep line algorithm for the straight\& line segment intersection problem. 1994.
- [2] Dan Sunday. The convex hull of a 2d point set or polygon. *ht—tp://softsurfer.com/Archive/algorithm*, 109, 2006.

Appendix:

Pygame is a popular Python library used for creating video games and multimedia applications. It's built on top of the Simple DirectMedia Layer (SDL) library, providing developers with functions and tools to work with graphics, sound, input devices, and more. Pygame simplifies game development by offering modules for handling graphics, events, sound, and input, allowing users to focus on game logic and design. It's versatile, enabling the creation of 2D games and multimedia applications across different platforms. With Pygame, developers can access and manipulate images, create animations, manage user input, and implement game physics, making it a powerful tool for game development in Python.

`pygame.font.Font()`, you can then use this object to render text onto surfaces within your game or application using methods like `render()` In Pygame, `blit()` is a method used to draw or transfer the contents of one surface onto another. It stands for "Block Transfer," and it's commonly used to render images, text, or other surfaces onto the game window or other surfaces within your Pygame application. `pygame` documentation = <https://www.pygame.org/docs/> `time` documentation = <https://docs.python.org/3/library/time.html> `math` documentation = <https://docs.python.org/3/library/math.html> `matplotlib` documentation = <https://matplotlib.org/stable/index.html>